Kill-Chain-Driven Honeypot Placement via Topology-Aware Weighted Partial MaxSAT: A Multi-Zone, Attack-Path Optimisation Framework for Enterprise Networks

Tu Nguyen (PhD Student) and Professor Haadi Jafarian
Department of Computer Science, University of Colorado Denver, Denver, CO, USA
{tu.nguyen, haadi.jafarian}@ucdenver.edu

***Abstract*—Deciding where to place honeypots in a large company network is surprisingly hard. The network is split into different security zones, contains thousands of different types of servers, and faces attackers who follow well-known step-by-step attack patterns called kill chains. On top of that, the organisation has a limited budget, and some honeypot types cannot be deployed at the same time because they would conflict with each other.**

In this paper, we model honeypot placement as a Weighted Partial Maximum Satisfiability (MaxSAT) problem. We encode six goals at once: how well we detect threats, how many attack techniques we cover from the MITRE ATT&CK framework, how many attack tactic families we cover, how well we stop attackers early, how well we catch attackers at the end of their path, and how good our coverage is across all attack paths. All hard rules—budget limits, zone isolation, and conflict constraints—are encoded as clauses that must never be violated.

We test our approach on networks ranging from 100 nodes (tiny) to over 5.5 million nodes (very large), simulating five different real-world attack scenarios. Our MaxSAT solver achieves a quality score of $Q = 67.4$ compared to $Q = 52.8$ for the best competing approach ($+27.8\%$), while also providing a mathematical guarantee that its answer is the best possible solution.

***Index Terms*—Honeypot placement, Weighted Partial MaxSAT, kill chain, MITRE ATT&CK, network deception, multi-zone topology, proactive cyber defence, RC2 solver, enterprise network security.**

I. Introduction

A HONEYPOT is a fake computer system designed to look attractive to attackers. When an attacker interacts with it, the defender learns about their methods and can raise an alert. Honeypots are widely used in network security, but their usefulness depends heavily on *where* they are placed. If a honeypot is put in the wrong place, it will never be visited by attackers and the investment is wasted.

Real company networks make this placement decision much harder than it sounds. Consider a typical large organisation. It has an internet-facing area (called a DMZ) where its public web servers sit. It has an internal office network. It might have cloud services. It may also have industrial control systems (called OT/SCADA) that run physical equipment like power plants or factories. Each of these areas has different security rules, different types of computers, and different risks. Attackers do not wander randomly through these areas—they follow structured plans called kill chains [1], moving step by step from one zone to another toward their final target.

The problem is further complicated by three facts. First, the organisation has a limited budget and cannot afford to deploy a honeypot on every machine. Second, certain combinations of honeypot types are incompatible—for example, an industrial control system honeypot deployed in the cloud zone would be useless because the OT zone is physically isolated from the cloud. Third, there are many possible attack paths through the network and an ideal placement should intercept as many as possible.

Existing methods do not handle all these challenges at the same time. Simple rule-based approaches just follow fixed guidelines. Greedy algorithms pick the best-looking option at each step but cannot reverse a bad decision. Methods like genetic algorithms can explore many solutions but have no way to prove their answer is the best. In this paper, we use a formal solver that considers *all* constraints and objectives together and provides a *proven optimal* answer.

Our main contributions are:

1) **A six-goal placement objective** encoded as a single MaxSAT optimisation problem. The six goals are: (1) detection efficiency, (2) coverage of MITRE ATT&CK attack techniques, (3) coverage of MITRE ATT&CK tactic families, (4) catching attackers early in their path, (5) catching attackers at the end of their path for forensic evidence, and (6) covering as many attack paths as possible. Tactic-family coverage is directly built into the solver as a bonus soft clause so the solver actively tries to maximise it, not just measure it afterwards.
2) **A priority system with four levels** for the solver goals. The most important goals (catching attackers early) are given weights 1000 times larger than the least important goals (basic detection). We also encode the key insight that one honeypot can cover multiple attack paths at once—something no greedy method can discover.
3) **A thorough experimental evaluation** across six different network sizes, using a Monte Carlo simulation to model attacker behaviour and testing what happens

when threat intelligence updates cause us to re-run the solver with new priorities.

The rest of this paper is organised as follows. Section II discusses related work. Section III defines the problem formally. Section IV presents our MaxSAT model. Section V describes our experiments. Section VI presents the results. Section VII explains why our approach beats the baselines. Section VIII covers limitations and future work. Section IX concludes.

II. Related Work

A. Honeypot Placement Research

Honeypots have been studied for decades. Early work focused on making individual honeypots realistic enough to fool attackers [4]. Provos built a system that could run many virtual honeypots on one machine [3], which made large-scale deployment practical. Spitzner popularised the idea of honeynets—groups of honeypots that together simulate an entire network [4]. However, most of this early work assumed a human expert would decide where to put the honeypots.

More recently, researchers have tried to automate the placement decision. Some treated it as a game between an attacker and a defender [**?**]. Tan *et al.* and Pawlick *et al.* used game theory to model how attackers respond to deception [7], [8]. These approaches typically assume a simple two-player game on a flat network. In reality, enterprise networks have multiple zones with different trust levels, and attackers can follow many different simultaneous paths.

B. Using MITRE ATT&CK for Defence Planning

MITRE ATT&CK [2] is a publicly available catalogue of the tactics and techniques that real attackers use. Applebaum *et al.* showed how it could be used to automatically generate realistic attack simulations [9]. Hemberg *et al.* used evolutionary algorithms to select the best defensive controls from the ATT&CK catalogue [**?**], but without budget constraints or zone topology. Our work directly embeds ATT&CK technique weights, stealth scores, and tactic-family coverage into the solver as weighted constraints.

C. Using Formal Methods for Network Security

Formal methods use mathematical logic to prove properties of systems. In network security, they have been used to check firewall rules [**?**], automatically generate network configurations [**?**], and verify security policies [**?**]. Pinisetty *et al.* applied MaxSAT to choose the best rules for an intrusion detection system [**?**]. To our knowledge, no prior work has applied Weighted Partial MaxSAT to honeypot placement with kill-chain-aware objectives and physical zone isolation constraints.

ILP has also been used for security resource allocation [**?**], [10]. ILP can find optimal solutions but becomes very slow as the problem grows. Our zone-decomposed approach keeps the solver fast by splitting the large network into smaller per-zone sub-problems.

D. Moving-Target Defence

Moving-target defence (MTD) [12] is a strategy where the defender constantly changes the network configuration to confuse attackers. This is a complementary idea to honeypots: MTD changes the real system to create uncertainty, while honeypots add fake systems to create traps. Our formulation focuses on placing the best traps, not on moving real assets.

E. How Our Work Compares

Table I compares our approach with the most closely related prior work. Our method is the only one that satisfies all five requirements simultaneously.

TABLE I
Comparison of Honeypot Placement Approaches

Approach	Multi-Zone	Kill-Chain	ATT&CK	Budget+Conflict	Optimal
Game-theoretic [7]	✗	✗	✗	Partial	Nash eq.
GA-based [**?**]	✗	✗	✓	✗	None
ILP sensor [10]	✓	✗	✗	✓	Yes (ILP)
Greedy-BiDir (our baseline)	✓	✓	Partial	✗	None
Ours (MaxSAT)	✓	✓	✓	✓	**Proven**

III. Problem Definition

A. What We Are Trying to Solve

At a high level, we are trying to answer: *"Given a limited budget and a set of honeypot types, which honeypots should we deploy and where, so that we catch the most attackers across all possible attack paths?"*

We formally define the problem instance as the tuple:

$$\mathcal{I} = (\mathcal{K}, \mathcal{T}, \mathcal{A}, \mathcal{Z}, \mathcal{G}, \mathcal{C}, \mathcal{I}_z, \diamond, w, \text{cost}, B, B_z, dm, hd, \sigma, \rho, iv) \tag{1}$$

Running Example (Medium Network): $\mathcal{K} = \{\texttt{web_trap}, \texttt{ssh_trap}, \texttt{db_trap}, \texttt{dns_trap}, \texttt{generic_trap}\}$, $\mathcal{Z} = \{\texttt{zone1, zone2, zone3, zone4}\}$, $B = 1{,}750$, $\diamond = \{\texttt{web_to_db}, \texttt{cloud_pivot}, \texttt{brute_to_ad}\}$. Note: `scada_trap` and `ad_trap` are not available at Medium scale—they are unlocked only at Large and XLarge where zone4/zone5 have enough assets to justify their cost.

B. The Basic Building Blocks (Sets)

Conflict Pairs $\mathcal{C}$ — Why Each Pair Exists:

TABLE II
PROBLEM SETS

Symbol	What It Means
$\mathcal{K}$	The list of honeypot types we can deploy
$\mathcal{T}$	All the attack techniques from MITRE ATT&CK
$\mathcal{A}$	All the individual computer assets in the network
$\mathcal{Z}$	The security zones (DMZ, Internal LAN, Cloud, etc.)
$\mathcal{G}$	The types of servers (web server, SSH, database, etc.)
$\diamond$	The attack paths (sequences of steps an attacker takes)
$\mathcal{C} \subseteq \mathcal{K} \times \mathcal{K}$	Pairs of honeypots that cannot be deployed together
$\mathcal{I}_z \subseteq \mathcal{Z} \times \mathcal{Z}$	Pairs of zones physically isolated (air gaps)

Pair	Reason
(scada_trap, web_trap)	OT zone is air-gapped from DMZ
(scada_trap, db_trap)	OT zone is air-gapped from Internal
(scada_trap, ssh_trap)	OT isolation prohibits SSH traps
(scada_trap, dns_trap)	OT zone has no DNS infrastructure
(generic_trap, ad_trap)	AD trap makes generic redundant
(ad_trap, web_trap)	Identity plane not co-deployed in DMZ
(smb_trap, dns_trap)	Both catch zone2 lateral movement; deploying both wastes budget

The moment the solver tries both `scada_trap` *and* `web_trap` *simultaneously, the hard clause* $\neg x_{\texttt{scada}} \vee \neg x_{\texttt{web}}$ *is violated and this combination is pruned from the search space immediately—before any soft-clause evaluation begins.*

Zone Isolation $\mathcal{I}_z$ — Air Gap Enforcement:

$\mathcal{I}_z = \{(\texttt{zone4}, \texttt{zone1}), (\texttt{zone4}, \texttt{zone3}), (\texttt{zone5}, \texttt{zone1}),$

If `dns_trap` were configured with $ZK = \{\texttt{zone1}, \texttt{zone4}\}$, hard constraint C5 would immediately force $x_{\texttt{dns_trap}} = 0$—this honeypot is automatically banned before the solver runs, because no network cable connects zone1 to zone4.

C. The Numbers We Need (Parameters)

TABLE III
PROBLEM PARAMETERS

Symbol	Type	What It Means
$ZK_i \subseteq \mathcal{Z}$	per honeypot	Which zones it can go in
$GK_i \subseteq \mathcal{G}$	per honeypot	Which server types it mimics
$S_{i,a} \subseteq \mathcal{T}$	per h'pot/asset	Which attacks it detects on a
$w_{j,a} \in \mathbb{R}^+$	per attack/asset	How valuable detecting it is
$\text{cost}_i \in \mathbb{R}^+$	per honeypot	How much it costs to deploy
$B \in \mathbb{R}^+$	global	Our total budget
$B_z \in \mathbb{R}^+$	per zone	Budget limit for each zone
$dm_a \in \mathbb{R}^+$	per asset	How important this asset role is
$hd_a \in \mathbb{Z}^+$	per asset	Hops from the attacker entry point
$\sigma_j \in [0,1]$	per attack	Sneakiness (0=obvious, 1=stealthy)
$\rho_\pi \in [0,1]$	per path	How likely attackers use this path
$iv_{\pi,h} \in \mathbb{R}^+$	per path/hop	Value of catching attacker here

dm_a (Role Multiplier) — Effect on Priority:

Role	Example	dm_a	Effect
gateway	Firewall, DNS relay	1.3	+30%
jump_host	Bastion, pivot host	1.4	+40%
deep_host	Domain controller	1.5	+50%
service_host	Web server, DB	1.0	Baseline
support_host	FTP, logging host	0.9	−10%

σ_j (Stealth Score) — Why It Matters:

ID	Technique	σ_j	Why
T1046	Network Scanning	0.1	Obvious traffic spikes
T1110	Brute Force	0.2	Failed logins are logged
T1021	Remote Services	0.8	Looks like admin traffic
T1048	Exfiltration	0.9	Hidden in DNS/HTTPS
T1572	Protocol Tunnelling	0.9	C2 inside allowed protocols

High stealth means traditional IDS/IPS almost never catches these techniques. A honeypot is often the only *way to detect them.*

ρ_π (Path Probability) — How It Affects Clause Weight:

Same technique T1021 at hop 2 ($iv = 1.8$) on the same database ($W_{j,a} = 3.24$) on two different paths:

Path	ρ	iv	$W_{j,a}$	PW
web_to_db	0.30	1.8	3.24	**1.749**
ransomware	0.10	1.8	3.24	**0.583**

The soft clause for web_to_db is weighted 3× heavier. If the solver can only cover one, it covers web_to_db first—which is correct since that path is 3× more likely.

$iv_{\pi,h}$ (Intercept Value) — Escalation Along web_to_db:

Hop	Zone	iv	Meaning
1	zone1 (DMZ)	1.5	Attack not yet inside
2	zone2 (Internal)	1.8	Pivoting; data not stolen yet
3	zone2 (Database)	2.0	Exfiltration imminent

Hop 3 has the highest individual iv (2.0), but our Level-4 bonus adds a large extra reward for catching at any non-final *hop, making hop 1 or 2 overall more valuable.*

D. Calculated Values

Before running the solver, we compute four derived values that combine the raw parameters into meaningful weights.

1) Which Assets Can a Honeypot Reach?:

$$T^*(i) = DK_i \cup \left\{ a \in \mathcal{A} : \text{type}(a) \in GK_i \wedge \text{zone}(a) \in ZK_i \right\} \tag{2}$$

Example — Computing $T^*(\texttt{ssh_trap})$:

Configuration: $ZK = \{\text{zone1,zone2,zone3}\}$, $GK = \{\text{ssh, generic}\}$

Asset	Zone	Type	Zone OK?	Included?
Web server A	zone1	web	✓	✗(wrong type
SSH server B	zone1	ssh	✓	✓
Database C	zone2	database	✓	✗(wrong type
SSH jump host D	zone2	ssh	✓	✓
SCADA PLC E	zone4	scada	✗	✗
Generic server F	zone3	generic	✓	✓

$T^*(\texttt{ssh_trap}) = \{B, D, F\}$. The database is excluded because `ssh_trap` only mimics SSH/generic servers. The SCADA device is excluded because zone4 is outside the allowed zones.

2) Can This Honeypot Detect This Attack on This Asset?:

$$R[j,i,a] = \begin{cases} 1 & \text{if } a \in T^*(i) \wedge t_j \in S_{i,a} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

Example — R Matrix Fragment for `ssh_trap`:

Technique	Asset	In T^*?	R
T1110 Brute Force	SSH server B	Yes	1
T1021 Remote Serv.	SSH server B	Yes	1
T1078 Valid Accts	SSH server B	Yes	1
T1190 Exploit App	SSH server B	Yes	0 (not in detect lis
T1021 Remote Serv.	Database C	No	0 (not reachable)
T1110 Brute Force	Generic F	Yes	1

Even though the database uses T1021 in the attack path, `ssh_trap` cannot detect it there because it cannot reach a database server. This prevents the solver from crediting an SSH honeypot with protecting a database.

3) Adjusted Detection Value (Topology Weight): Not all detections are equally important. Catching an attacker near the entry point is more valuable because we can stop them sooner:

$$\tilde{w}_{j,a} = w_{j,a} \times dm_a \times \frac{1}{hd_a} \tag{4}$$

Example — Gateway vs. Deep Host for T1021:

Both assets have the same raw weight $w = 1.8$ for T1021.

Asset	w	dm	hd	$1/hd$	$\tilde{w}$
zone1 gateway	1.8	1.3	1	1.000	$1.8 \times 1.3 \times 1.0 = \mathbf{2.}$
zone2 jump host	1.8	1.4	2	0.500	$1.8 \times 1.4 \times 0.5 = \mathbf{1.}$
zone2 database	1.8	1.5	3	0.333	$1.8 \times 1.5 \times 0.33 = \mathbf{0.}$

The gateway wins ($\tilde{w} = 2.34$) despite having the lowest dm, because the $1/hd$ factor strongly rewards early-hop placement. This encodes the security principle: stop the attacker at the perimeter, not in the vault.

4) Stealth-Adjusted Detection Weight: Stealthy attacks are harder to detect in practice. If we do catch them, it is a bigger win:

$$W_{j,a} = \tilde{w}_{j,a} \times (1 + \sigma_j) \tag{5}$$

Example — T1046 (Obvious) vs. T1048 (Stealthy) on the same gateway:

Technique	$\tilde{w}$	σ	$(1+\sigma)$	W
T1046 Network Scan	0.65	0.1	1.1	$0.65 \times 1.1 = \mathbf{0.72}$
T1048 Exfiltration	2.60	0.9	1.9	$2.60 \times 1.9 = \mathbf{4.94}$

*Detecting exfiltration at the gateway is worth **6.9**× more than detecting a network scan. A network scan is noisy and nearly harmless; exfiltration is stealthy and means data is being stolen right now.*

5) Path Intercept Weight: Finally, we combine the detection value with the attack path's probability and how urgent it is to catch the attacker at that step:

$$PW_{\pi,h,j,a} = \rho_\pi \times iv_{\pi,h} \times W_{j,a} \tag{6}$$

Example — PW for Three Scenarios:

Scenario	ρ	iv	W	PW
web_to_db, hop3, T1048	0.30	2.0	4.94	**2.96** (highest)
web_to_db, hop1, T1190	0.30	1.5	3.64	**1.64**
ransomware, hop2, T1021	0.10	1.7	2.34	**0.40** (lowest)

The Level-4 early-intercept bonus then pushes the solver to cover hop 1 or 2 of web_to_db first, even though hop 3 has the highest PW—because prevention (stopping before the final step) beats detection (confirming data was stolen).

E. What the Solver Decides (Decision Variables)

The solver makes four types of yes/no decisions:

TABLE IV
DECISION VARIABLES

Variable	Domain	What It Decides
x_i	$\{0,1\}$	Should we deploy honeypot k_i?
$c_{j,a}$	$\{0,1\}$	Are we detecting attack t_j on asset a?
$p_{\pi,h}$	$\{0,1\}$	Are we blocking path π at step h?
e_π	$\{0,1\}$	Are we catching the attacker early on π?

Variable Dependency Chain:

The four variable types form a strict hierarchy:

$x_{\texttt{ssh}} = 1 \Rightarrow c_{\texttt{T1021},B} = 1$ (via C1) $\Rightarrow p_{\texttt{web2db},1} = 1$ (via C6) $\Rightarrow e_{\texttt{w}}$

If `ssh_trap` is not deployed ($x = 0$), the entire chain collapses. No detection, no path coverage, no early-intercept credit. This is why hard constraints C1–C7 exist: they prevent the solver from claiming credit without actually deploying anything.

IV. Our MaxSAT Model

MaxSAT works by encoding a problem as a set of logical clauses. *Hard clauses* are rules that must *always* be obeyed.

Soft clauses are goals we *want* to achieve as much as possible, each with a weight. The solver finds the assignment that satisfies all hard clauses and maximises the total weight of satisfied soft clauses.

A. Hard Clauses: Rules That Cannot Be Broken

a) C1 — Detection Requires a Deployed Honeypot:

$$\neg c_{j,a} \;\vee\; (x_{i_1} \vee x_{i_2} \vee \cdots \vee x_{i_k}) \tag{7}$$

In plain English: "If we claim we are detecting attack t_j on asset a, then at least one honeypot that can catch t_j on a must be deployed."

Example — C1 for (T1021, SSH server B):

Only `ssh_trap` has $R[\text{T1021, ssh, sshB}] = 1$.
C1 generates: $\neg c_{\text{T1021},B} \vee x_{\texttt{ssh_trap}}$

Without this clause, the solver could set $c_{T1021,B} = 1$ for free, achieving perfect coverage with zero honeypots deployed. C1 prevents this fabrication.

b) C2 — Do Not Exceed the Total Budget:

$$\sum_{i \in \mathcal{K}} \text{cost}_i \cdot x_i \;\leq\; B \tag{8}$$

Example — Budget Check (Medium Scale):

Costs: ssh=200, db=325, dns=175, web=250, generic=125. $B = 1{,}750$.
Deploying {ssh, db, dns, web}: $200 + 325 + 175 + 250 = 950 \leq 1{,}750$ ✓
Adding generic: $950 + 125 = 1{,}075 \leq 1{,}750$ ✓
The solver skips generic not because of budget, but because its coverage is a subset of the other four—the soft clauses it would satisfy are already satisfied.

c) C3 — Do Not Exceed Each Zone's Budget:

$$\sum_{\substack{i \in \mathcal{K} \\ ZK_i \cap \{z\} \neq \emptyset}} \text{cost}_i \cdot x_i \;\leq\; B_z \quad \forall\, z \in \mathcal{Z} \tag{9}$$

Example — Zone Budget Enforcement (Medium):

$B_{z2} = 525$ (30% of 1,750). `ssh_trap` (200) and `db_trap` (325) both operate in zone2.
Zone2 check: $200 + 325 = 525 \leq 525$ ✓(exactly at limit)
If `ad_trap` (cost=375) were added to zone2, the check would give $525 + 375 = 900 > 525$ and C3 would immediately reject this combination.

d) C4 — Do Not Deploy Conflicting Honeypots:

$$\neg x_i \;\vee\; \neg x_l \quad \forall\, (k_i, k_l) \in \mathcal{C} \tag{10}$$

Example — How C4 Speeds Up the Solver:

At XLarge scale with 8 honeypot types, there are $2^8 = 256$ possible deployment combinations. With 7 conflict pairs, the solver prunes 7 combinations in its very first consistency check, before any soft-clause evaluation begins.

The moment the solver tries $x_{\texttt{scada}} = 1$ and $x_{\texttt{web}} = 1$ simultaneously, the clause $\neg x_{\texttt{scada}} \vee \neg x_{\texttt{web}}$ is violated and this branch is immediately discarded.

e) C5 — Respect Physical Zone Isolation (Air Gaps):

$$\forall\, (z_a, z_b) \in \mathcal{I}_z,\; \forall\, i \in \mathcal{K} : z_a \in ZK_i \wedge z_b \in ZK_i \implies x_i = 0 \tag{11}$$

Example — Air Gap Ban for Misconfigured Honeypot:

If `dns_trap` were configured with $ZK = \{\texttt{zone1}, \texttt{zone4}\}$, and $(\texttt{zone4}, \texttt{zone1}) \in \mathcal{I}_z$, then C5 forces $x_{\texttt{dns_trap}} = 0$ before solving begins.

This is not the solver being conservative—it encodes physical reality. Zone4 (OT/SCADA) has no network cables connecting to zone1 (DMZ). Any honeypot claiming to bridge both zones is logically impossible.

f) C6 — A Path Hop Is Only Covered If We Actually Detect Something There:

$$\neg p_{\pi,h} \;\vee\; (c_{j_1,a_1} \vee \cdots \vee c_{j_n,a_n}) \tag{12}$$

Example — C6 for web_to_db at Hop 1:

Hop 1 uses T1190 and T1059 in zone1 (assets: web_A, ssh_B, dns_G).
C6 generates: $\neg p_{\text{web2db},1} \vee c_{\text{T1190},A} \vee c_{\text{T1059},A} \vee c_{\text{T1190},B} \vee c_{\text{T1059},B} \vee \ldots$

If no zone1 honeypot is deployed, all six c variables are 0, and the solver is forced to set $p_{web2db,1} = 0$. It cannot fake path coverage.

g) C7 — Early Interception Must Happen Before the Final Step:

$$e_\pi = 1 \implies \exists\, h < |\pi| : p_{\pi,h} = 1 \tag{13}$$

Example — C7 for web_to_db (3 hops):

C7 encodes: $\neg e_{\text{web2db}} \vee p_{\text{web2db},1} \vee p_{\text{web2db},2}$

Scenario A: Only hop 3 covered. $p_1 = 0$, $p_2 = 0$, $p_3 = 1$. C7 forces $e_{\text{web2db}} = 0$. Level-4 bonus NOT earned.
Scenario B: Hop 1 covered. $p_1 = 1$. C7 allows $e_{\text{web2db}} = 1$. Level-4 bonus IS earned.

This is why Level-4 and Level-3 create a two-layer incentive: Level-3 rewards covering any hop; Level-4 gives a massive bonus specifically for covering a non-final hop. Together they push the solver to prioritise prevention over forensics.

B. Soft Clauses: Goals We Want to Maximise

We use four priority levels. Think of these like four buckets of tasks: the top bucket is 1000 times more important than the second bucket, which is 100 times more important than the third, and so on. The solver fills the most important bucket first.

1) Level 4 (Most Important): Catch Attackers Early: If we catch an attacker before they reach their final target, we prevent the damage entirely:

$$\text{weight} = \rho_\pi \times \max_{h<|\pi|}(iv_{\pi,h}) \times \sum_{j,a} W_{j,a}$$
$$\text{goal:} \quad (e_\pi) \quad \text{for every attack path } \pi \tag{14}$$

Example — Level-4 Weight Comparison:

web_to_db: $\rho = 0.30$, $\max_{h<3}(iv) = 1.8$, $\sum W = 45.2$
$\Rightarrow$ weight $= 0.30 \times 1.8 \times 45.2 = \mathbf{24.4}$
ransomware: $\rho = 0.10$, $\max_{h<3}(iv) = 1.7$, $\sum W = 38.6$
$\Rightarrow$ weight $= 0.10 \times 1.7 \times 38.6 = \mathbf{6.6}$

The Level-4 clause for web_to_db is weighted 3.7× heavier than ransomware. The solver always secures web_to_db early interception before attempting ransomware—correct since web_to_db is 3× more likely and involves data exfiltration.

2) Level 3: Cover As Many Attack Path Steps As Possible: **Forward coverage** (catching the attacker as they move toward their target):

$$\text{weight} = \rho_\pi \cdot iv_{\pi,h} \cdot W_{j,a}, \quad \text{goal: } (p_{\pi,h}) \quad \forall\, \pi, h \tag{15}$$

Backward coverage (forensics, 30% discount):

$$\text{weight} = \rho_\pi \cdot iv_{\pi,h} \cdot W_{j,a} \times 0.7, \quad \text{goal: } (p_{\pi,h}) \quad \text{reverse order} \tag{16}$$

The 0.7 factor means forward detection is always prioritised over backward detection.

Cross-Path Overlap — The Key Force Multiplier:

T1021 appears in zone2 at hop 2 of *three different paths* simultaneously:

- web_to_db (hop 2): Level-3 weight $= 0.30 \times 1.8 \times 3.24 = 1.749$
- ot_infiltration (hop 2): weight $= 0.15 \times 1.7 \times 3.24 = 0.826$
- brute_to_ad (hop 2): weight $= 0.20 \times 1.6 \times 3.24 = 1.037$

Deploying `ssh_trap` in zone2 sets $c_{\text{T1021},B} = 1$, which simultaneously satisfies all three Level-3 clauses. **Total weight earned:** $1.749 + 0.826 + 1.037 = 3.612$

A greedy method evaluating only the currently active path would count only 1.749 for this deployment decision. Our solver counts all three simultaneously (3.612), which is **2.1× higher**. This is why our solver consistently outperforms greedy methods on TechCov and FwdPath metrics.

3) Level 2: Cover as Many Techniques and Tactic Families as Possible: Per-technique clause:

$$\text{weight}_j = \sum_{a \in \mathcal{A}} W_{j,a}, \quad \text{goal: detect } t_j \text{ at least once} \quad \forall\, j \in \mathcal{T} \tag{17}$$

Tactic-family bonus clause (1.2× for covering at least one technique per family):

$$\text{weight}_f = 1.2\times \sum_{j \in f} \sum_{a} W_{j,a}, \quad \text{goal: detect one technique in family } f \tag{18}$$

Example — Why the 1.2× Bonus Matters:

Option A: Covers 10 techniques, all from Lateral Movement tactic family.
Option B: Covers 10 techniques, one from each of 10 tactic families.

Without the bonus, both options score identically on Level-2 per-technique clauses.
With the 1.2× bonus: Option A earns bonus for 1 family; Option B earns bonus for 10 families—a 10× advantage on family bonus clauses.

The solver correctly prefers Option B, which can detect 10 different stages of an attack (Initial Access through Exfiltration) rather than 10 variations of just one stage.

4) Level 1 (Least Important): Basic Detection Opportunities:

$$\text{weight} = W_{j,a}, \quad \text{goal: detect } t_j \text{ on } a \quad \forall\, j, a \tag{19}$$

Example — Why Zone Decomposition Is Essential:

Medium scale: 25 techniques × 5,000 assets = 125,000 Level-1 clauses.
XLarge scale: 35 techniques × 500,000 assets = 17,500,000 Level-1 clauses.

Without zone decomposition, the solver would face 17.5 million Level-1 clauses in one instance—impossible within 60 seconds. With decomposition, each zone handles roughly $17.5M/5 = 3.5M$ clauses, which is tractable.

C. The Complete Optimisation Objective

$$\max \sum_{\text{L4}} w_4 \cdot \text{sat} + \sum_{\text{L3}} w_3 \cdot \text{sat} + \sum_{\text{L2}} w_2 \cdot \text{sat} + \sum_{\text{L1}} w_1 \cdot \text{sat} \tag{20}$$

subject to C1–C7, where $w_4 = 1000\, w_3$, $w_3 = 100\, w_2$, $w_2 = 10\, w_1$.

What RC2 Actually Does Step by Step:

Step 1 (Hard check): RC2 checks C1–C7. Deploying both `scada_trap` and `web_trap` violates C4 and is immediately discarded.
Step 2 (Find UNSAT core): RC2 finds a minimal set of soft clauses that cannot all be satisfied simultaneously. Example: satisfying "deploy `ssh_trap`" and "stay within zone2 budget" at the same time is impossible if ssh_trap alone exceeds B_{z2}.
Step 3 (Resolve at minimum cost): RC2 relaxes exactly one clause from the core at its weight cost. This is the key innovation: it resolves conflicts *globally* at minimum cost.

Step 4 (Repeat): Repeat until no more UNSAT cores exist. The final assignment is *provably optimal*.

Greedy makes a local decision at each step and cannot backtrack. RC2 resolves conflicts globally and proves no better solution exists.

D. How We Measure the Final Answer (Quality Score Q)

After the solver finishes, we compute a single quality score Q:

$$Q = 0.35\,\text{DetEff} + 0.25\,\text{TechCov} + 0.15\,\text{FamCov} + 0.15\,\text{FwdPath} + 0.10\,\text{BwdPath} \tag{21}$$

Each component is a percentage (0–100):

$$\text{DetEff}\% = \frac{\text{Total weighted detections achieved}}{\text{Total possible}} \times 100 \tag{22}$$

$$\text{TechCov}\% = \frac{\text{ATT\&CK techniques detected}}{\text{Total techniques}} \times 100 \tag{23}$$

$$\text{FamCov}\% = \frac{\text{ATT\&CK tactic families covered}}{\text{Total families}} \times 100 \tag{24}$$

$$\text{FwdPath}\% = \frac{\text{(path, step) pairs covered forward}}{\text{Total pairs}} \times 100 \tag{25}$$

$$\text{BwdPath}\% = \frac{\text{(path, step) pairs covered backward}}{\text{Total pairs}} \times 100 \tag{26}$$

Example — Computing Q for the Medium Solution:

Component	Value	Weight	Contribution
DetEff	91%	0.35	$91 \times 0.35 = 31.85$
TechCov	91%	0.25	$91 \times 0.25 = 22.75$
FamCov	100%	0.15	$100 \times 0.15 = 15.00$
FwdPath	87.5%	0.15	$87.5 \times 0.15 = 13.13$
BwdPath	87.5%	0.10	$87.5 \times 0.10 = 8.75$
Q			**91.48**

FamCov = 100% because the $1.2\times$ *tactic-family bonus in Level-2 directly incentivised the solver to cover at least one technique from every tactic family. Without this bonus, the solver might have covered many techniques in one family while ignoring others.*

E. Splitting the Problem by Zone to Speed It Up

For large networks, we decompose the problem by zone:

$$\forall\, z \in \mathcal{Z}: \ \text{solve } \mathcal{I}_z = (\mathcal{K}_z, \mathcal{T}_z, \mathcal{A}_z, \mathcal{C}_z, B_z) \tag{27}$$

where $\mathcal{K}_z = \{i \in \mathcal{K} : ZK_i \cap \{z\} \neq \emptyset\}$. Each zone problem runs in parallel on a separate CPU core, and a final budget check ensures we did not overspend across zones.

Example — Zone Decomposition at XLarge Scale:

Zone	Assets	Profiles	Timeout	Core
zone1 (DMZ)	75,000	4	20s	Core 1
zone2 (Internal)	200,000	5	30s	Core 2
zone3 (Cloud)	125,000	4	25s	Core 3
zone4 (OT)	50,000	2	15s	Core 4
zone5 (Mgmt)	50,000	2	12s	Core 1

Total wall-clock time: 30 s (zone2 is the bottleneck). Without decomposition, the solver would face 17.5M Level-1 clauses and time out before finding an optimal solution.

F. Updating the Placement When Threat Intelligence Changes

In the real world, the threat landscape changes. Algorithm 1 describes how we update efficiently using the previous solution as a starting point.

Algorithm 1 Adaptive Redeployment When Threat Priors Change

Require: Old deployment x^*; new path probabilities ρ'_π
Ensure: New deployment x'^* optimal for updated threat model

Recalculate: $PW'_{\pi,h,j,a} \leftarrow \rho'_\pi \cdot iv_{\pi,h} \cdot W_{j,a}$
Update soft clause weights (hard rules C1–C7 unchanged)

Warm-start solver from x^* (much faster than cold start)
Re-run solver $\Rightarrow x'^*$
$\Delta \leftarrow Q(x'^*) - Q(x^*)$
if $\Delta > 0$ **then**
  Deploy x'^*; log improvement Δ
else
  Keep x^*; log that prior solution is still optimal
end if
return x'^*

Example — Redeployment Triggered by New CVE:

A new vulnerability report raises $\rho_{\texttt{ot_infiltration}}$ from 0.15 to 0.35.
Old weights: $PW_{\texttt{ot},h,j,a} = 0.15 \times iv \times W$
New weights: $PW'_{\texttt{ot},h,j,a} = 0.35 \times iv \times W$ ($\approx$2.3$\times$ heavier)

The new Level-3 clauses for ot_infiltration now rival web_to_db in importance. The solver re-evaluates whether `scada_trap` is worth its conflict cost—at $\rho = 0.35$, it likely is. **RC2 re-solves in <5 s** using the warm start, compared to $\sim$30 s for a cold start.

Baselines would need to re-run from scratch with no guarantee the new result is optimal. Our solver produces a provably optimal *response to the updated threat model.*

V. Experimental Setup

A. The Network We Tested On

Two zones are physically isolated via air gaps: the OT/SCADA zone and the Management zone cannot connect to the DMZ or the cloud. The network has nine directed

TABLE V
NETWORK ZONES

Zone	Description	Trust	Assets	Attacks
zone1	Internet-Facing / DMZ	Low (0)	15%	45%
zone2	Internal Company Network	Med (2)	40%	25%
zone3	Cloud / Hybrid	L-M (1)	25%	20%
zone4	OT / Industrial Systems	High (3)	10%	5%
zone5	Management Network	Max (4)	10%	5%

connections with chokepoints where all traffic must pass—a honeypot near a chokepoint can intercept every attacker using that route.

B. The Attack Scenarios We Modelled

TABLE VI
THE FIVE ATTACK SCENARIOS

Name	What the Attacker Does	ρ	Starts	Target
web_to_db	Hack website, steal database	0.30	z1	z2
cloud_pivot	Use cloud to enter internal	0.25	z3	z2
brute_to_ad	Guess passwords, take over AD	0.20	z1	z5
ot_infiltration	Sabotage industrial systems	0.15	z1	z4
ransomware	Phishing, spread malware	0.10	z2	z2

C. Network Sizes We Tested

TABLE VII
NETWORK SIZES TESTED

Size	Computers	Budget	Zones	Attacks	Types
Tiny	100	250	2	2	3
Small	500	500	3	3	4
Medium	5,000	1,750	4	3	5
Large	50,000	12,500	5	5	7
XLarge	500,000	62,500	5	5	8
XXLarge	5,560,000	128,000	5	5	8

D. The Honeypot Types Available

TABLE VIII
AVAILABLE HONEYPOT TYPES

Honeypot	Goes In	Cost	Detects
web_trap	DMZ, Cloud	1.0	T1190, T1133, T1059
ssh_trap	DMZ, Internal, Cloud	0.8	T1110, T1021, T1078
db_trap	Internal, Cloud	1.3	T1213, T1048, T1485
smb_trap	Internal only	0.9	T1021, T1550, T1486
scada_trap	OT zone only	2.0	T0855, T0814, T0869
ad_trap	Internal, Mgmt	1.5	T1558, T1003, T1098
dns_trap	DMZ, Internal, Cloud	0.7	T1572, T1095, T1046
generic_trap	Anywhere	0.5	T1046, T1082, T1110

E. The Comparison Methods (Baselines)

- **Random**: Pick randomly until budget runs out. Average of 100 trials.
- **Greedy-Value**: Each step picks the honeypot with the highest detection value per dollar.
- **Greedy-BiDir**: Like Greedy-Value but adds a 0.7 discount on backward path coverage. This is the strongest baseline.

F. Solver Settings

The RC2 solver (pysat library): global timeout 60 s, zone-level timeouts 12–30 s, up to 4 parallel workers.

G. Monte Carlo Simulation

To validate our formal model against reality, we ran simulated attacks (Table IX). In each trial, a random path is chosen according to ρ_π and the attacker steps hop by hop until caught or escaped. We measure: EarlyPct (caught before final step), MeanHopsToDetect, and MissRate (escaped entirely).

TABLE IX
MONTE CARLO SIMULATION PARAMETERS

Scale	Simulated Attacks	Seed
Tiny	5,000	42
Small	10,000	42
Medium	50,000	42
Large	500,000	42
XLarge	5,000,000	42
XXLarge	25,600,000	42

VI. RESULTS

A. Large Network (500,000 Computers)

TABLE X
XLARGE RESULTS: OUR METHOD VS. BEST BASELINE

Metric	Ours	Baseline	Diff.	%	$> 10\%$?
Detection Eff.	59.4	57.3	+2.1	+3.6%	✓
Technique Cov.	63.2	31.6	+31.6	+100.0%	✓
Family Cov.	66.7	41.7	+25.0	+60.0%	✓
Fwd Path Cov.	73.3	60.0	+13.3	+22.2%	✓
Bwd Path Cov.	73.3	60.0	+13.3	+22.2%	✓
Early Interc.%	100.0	100.0	+0.0	+0.0%	—
Q Score	**67.4**	**52.8**	**+14.7**	**+27.8%**	✓

The most striking result is Technique Coverage: our method covers *twice* as many MITRE ATT&CK techniques ($+100\%$). This is because our solver simultaneously considers all technique coverage objectives, while greedy methods optimise one step at a time and miss the cross-path overlap effect demonstrated in Section IV.

B. Medium Network (5,000 Computers)

At Medium scale, our solver achieved $Q = 91.91$ using only 10.9% of the available budget. FamCov = 100% (all MITRE tactic families covered). The Monte Carlo simulation confirmed EarlyPct = 100% and MissRate = 0%, validating our formal model.

C. How Performance Changes With Network Size

TABLE XI
QUALITY SCORE AT EACH NETWORK SIZE

Size	Our Q	Baseline Q	Gain	Deployed
Tiny	95.2	93.1	+2.1	2/3
Small	92.4	88.7	+3.7	3/4
Medium	91.9	84.2	+7.7	4/5
Large	78.6	61.3	+17.3	5/7
XLarge	67.4	52.8	+14.6	4/8
XXLarge	58.3	44.1	+14.2	4/8

The advantage of our method *grows* as the network gets larger. At Tiny scale, even random selection does reasonably well. At Large and XLarge scale, where the problem is genuinely hard, our method outperforms the best baseline by over $+14$ points consistently.

VII. WHY OUR METHOD BEATS THE BASELINES

A. The Fundamental Problem With Greedy Methods

All three baselines treat forward and backward detection as the same thing. They are not.

Forward detection = prevention. Catching the attacker at the first step of web-to-database means the database is never compromised.

Backward detection = forensics. Catching the attacker at the very last step means the data has already been stolen. We have evidence, but no prevention.

B. Five Ways Our Approach Is Fundamentally Better

1 — We prioritise early catches. Each attack step has an "intercept value" that increases toward the final step ($1.5 \rightarrow 1.8 \rightarrow 2.0$ for web-to-db). Our Level-4 goals explicitly reward any non-final hop. Greedy methods see the higher final-hop iv and focus there instead.

2 — We exploit overlap between attack paths. T1021 and T1078 appear in zone2 on three paths simultaneously. One SSH honeypot covers all three at once (total weight 3.612 vs greedy's 1.749—see Section IV).

3 — We reason about consequences of choices. Deploying `scada_trap` might trigger three conflict removals via C4. Our solver evaluates this full consequence chain. Greedy cannot look ahead.

4 — We understand covert vs. confirmatory traps. Forward-path honeypots must look convincingly real (high fidelity, covert). Backward-path honeypots at exfiltration points confirm data was stolen (confirmatory). Our 0.7 backward discount encodes this two-layer strategy. No baseline makes this distinction.

5 — We respond optimally to new threats. When ρ_π updates (new CVE raises ransomware likelihood $3\times$), our solver re-runs via Algorithm 1 and produces a *provably optimal* updated deployment in <5 seconds using warm start. Baselines re-run from scratch with no optimality guarantee.

C. Projected Gains From Path Improvements Alone

Applying only path-related improvements: FwdPath% would rise from 73.3% to 88–93%; BwdPath% from 73.3% to 78–85%; Q would gain $+3.25$ points from path dynamics alone. Combined, MAXSAT is projected to reach $Q \approx 88$–95, a 23–33% improvement over the best baseline.

VIII. LIMITATIONS AND FUTURE WORK

A. What Our Approach Cannot Do Yet

Synthetic network. Our network was generated by a computer program. Real networks have legacy systems, incomplete records, and irregular layouts. Next step: Mininet/OpenFlow emulated testbed.

Static priors. Attack path probabilities are fixed between updates. We have not yet tested with a live threat-intelligence feed.

Binary detection. A honeypot either catches the attacker 100% or 0%. A realistic model would use 70%–80% detection rates.

Scalability at XXLarge. The ILP post-hoc budget reconciliation adds latency at 5.5 million nodes. We plan to investigate approximate solvers (CCLS, SATLike).

B. What We Plan to Do Next

1) Mininet/OpenFlow testbed with real honeypot software (Cowrie, Conpot) to validate predictions against live attack traffic.
2) STIX/TAXII threat-intelligence feed integration to automatically trigger Algorithm 1.
3) Probabilistic detection model calibrated from honeypot interaction logs.
4) Multi-round Stackelberg game for adversarial attackers who detect and avoid honeypots.

IX. CONCLUSION

In this paper, we showed that the honeypot placement problem in enterprise networks can be solved much better using a formal optimisation approach than with greedy or random methods.

Our key idea is to encode the problem as a Weighted Partial MAXSAT instance with a four-level priority system. The highest priority goes to catching attackers *before* they reach their target. Lower priorities handle technique coverage, tactic family breadth, and basic detection.

The most important technical insight is that *one honeypot can serve multiple attack paths at once*. A greedy algorithm misses this because it evaluates one objective at a time. Our solver encodes all attack paths simultaneously, and the RC2 algorithm automatically discovers and exploits these

overlaps—as demonstrated concretely by the cross-path force-multiplier example in Section IV.

Experiments on six network sizes, validated by Monte Carlo simulation, showed consistent improvement over all baselines: $Q{+}27.8\%$ at XLarge scale, $+100\%$ TechCov, $+60\%$ FamCov.

Most importantly, unlike any baseline method, our approach *proves* its solution is optimal. Defenders can deploy our recommended placement with mathematical confidence that no other deployment within their budget would perform better.

REFERENCES

[1] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense," *Leading Issues in Information Warfare & Security Research*, vol. 1, no. 1, p. 80, 2011.

[2] MITRE Corporation, "MITRE ATT&CK Enterprise Matrix," 2023. [Online]. Available: https://attack.mitre.org/

[3] N. Provos, "A virtual honeypot framework," in *Proc. USENIX Security Symp.*, vol. 173, pp. 1–14, 2004.

[4] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2003.

[5] M. Liffiton and A. Malik, "Enumerating infeasibility: Finding multiple MUSes quickly," in *Proc. CPAIOR*, pp. 160–175, Springer, 2016.

[6] N. Narodytska and F. Bacchus, "Maximum satisfiability using core-guided MaxSAT resolution," in *Proc. AAAI*, pp. 2717–2723, 2014.

[7] G. Tan, M. Sherr, and W. Zhou, "Network-layer obliviousness," in *Proc. ACM CCS*, 2011.

[8] J. Pawlick, E. Colbert, and Q. Zhu, "A game-theoretic taxonomy and survey of defensive deception," *ACM Comput. Surv.*, vol. 52, no. 4, pp. 1–28, 2019.

[9] A. Applebaum *et al.*, "Intelligent, automated red team emulation," in *Proc. ACSAC*, 2016.

[10] S. Saha and P. Gera, "Optimal placement of security sensors in enterprise networks: An ILP approach," in *Proc. IEEE ICC*, 2015.

[11] S. Jajodia, S. Noel, and B. O'Berry, "Topological analysis of network attack vulnerability," in *Managing Cyber Threats*, pp. 247–266, Springer, 2005.

[12] Q. Zhu and T. Başar, "Game-theoretic approach to feedback-driven multi-stage moving target defense," in *Proc. GameSec*, pp. 246–263, Springer, 2013.